

TIB SYSTEMS LLC

SAP Security, GRC & Controls — Training Division

SAP Foundations Primer

A Prerequisite Study Guide for the SOX / ITGC Control Audit Course Track

Prepared for: Incoming College Graduates / No Prior IT Background

Document Code: TIB-TRN-PREREQ-SAP-FOUNDATIONS-001

Course Track: Prerequisite to SAP SOX / ITGC Control Audit Course Series

Version: 1.0

Issued: August 2026

WHO THIS GUIDE IS FOR

You are about to study how organizations use SAP to run finance, procurement, and operations — and how auditors test whether the controls inside SAP actually work. This guide assumes zero IT background. By the end, you will understand what a database, an application server, and a client are; how a screen in SAP turns into a saved business transaction; and what is really happening when someone runs a T-code.

How to Use This Guide

This primer is designed to be read once start-to-finish before you begin the SOX/ITGC Control Audit course track, and then kept as a reference while you work through it. It does not teach you to be a programmer or a system administrator. It teaches you enough about how enterprise software is built and how it runs so that terms like “application server,” “authorization object,” “table,” and “T-code” stop being jargon and start being things you can picture.

Controls auditing is fundamentally about answering one question, over and over, in different contexts: “can I trust that this system did what it was supposed to do, and only what it was allowed to do, and that there is a reliable record of it?” You cannot answer that question credibly without a working mental model of where data lives, who can touch it, and how a business transaction actually moves through the system. That is what this guide builds.

Study Approach

1. Read Chapters 1–3 first to get the big-picture architecture (the “three boxes” model).
2. Read Chapters 4–6 to understand each layer — database, application server, client — in more depth.
3. Read Chapter 7 slowly. This is the most important chapter: it walks through, step by step, what happens between a keystroke and a saved database record. Do not skim it.
4. Read Chapter 8 to connect that mechanical picture to what a T-code actually is.
5. Use Chapter 9 to see explicitly why every one of these technical concepts shows up again in the SOX/ITGC course — this is the bridge between “how SAP works” and “how we audit SAP.”
6. Use the Glossary (Chapter 10) as a running reference, and complete the Readiness Checklist (Chapter 11) before your first live course session.

Chapter 1 — The Big Picture: What Is an Enterprise Application?

Before SAP, before “the cloud,” before any of the vocabulary you are about to learn, every business software system is trying to solve the same three problems:

7. Store information reliably (customers, vendors, employees, financial postings, inventory).
8. Let many people work with that information at the same time, from different locations, without stepping on each other.
9. Enforce rules about who is allowed to see or change what — and keep a record of what happened.

SAP is one (very large, very configurable) answer to those three problems. It is what is called an ERP system — Enterprise Resource Planning — meaning a single software system that many different departments (Finance, Procurement, HR, Sales, Manufacturing) all use to run their part of the business, with the data flowing between departments automatically. When a warehouse clerk records that goods arrived, Finance's books can update without anyone re-typing anything.

Why this matters for auditing: an ERP is powerful precisely because so many things are connected and automated. That same power is exactly why controls matter so much — a single misconfigured setting or an inappropriate access right does not just affect one person's spreadsheet; it can affect every transaction that flows through that process, for every user, every day, until someone catches it.

Every Enterprise Application Is Really Three Things Working Together

No matter how complicated SAP looks on the screen, everything you will ever do in it is built from three physical/logical layers. Manufacturers describe this as a three-tier (or three-layer) architecture. Picture three separate boxes, usually three separate computers (sometimes many computers acting as one), each with a distinct job:

Layer	Its Job, in Plain English
Client Layer	The layer closest to the human being. This is what you see and touch — the SAP GUI window, a Fiori app in a browser, or a mobile app. It knows almost nothing on its own; its job is to display and collect information.
Application Server Layer	The “brain.” This is where the actual business logic lives and runs — the rules, the calculations, the step-by-step programs that decide what a transaction is allowed to do.
Database Server Layer	The “filing cabinet.” This is where every piece of data is permanently stored — every customer record, every posted journal entry, every user's access rights — organized into tables.

KEY IDEA

Nothing you type is ever “saved” at the client layer, and almost no business logic runs there. The client asks; the application server decides and does the work; the database remembers. Keep this three-box picture in your head — every chapter that follows is really just zooming into one of these three boxes.

Chapter 2 — The Three-Tier Architecture in Detail

SAP calls this pattern the “client-server architecture,” and specifically a three-tier client-server architecture, because there are three distinct tiers (layers) instead of just two. Older, simpler systems were two-tier: a client talked directly to a database. SAP deliberately inserted a middle layer — the application server — between the person and the data, and that decision is the single most important architectural fact to understand before you study controls.

2.1 Why a Middle Layer Exists at All

If users could connect straight to the database and type whatever they wanted, there would be no consistent way to enforce business rules (“you cannot post a journal entry that does not balance”), no consistent way to check authorizations (“you are not allowed to approve your own purchase order”), and no reliable audit trail. The application server exists to be the single, controlled gatekeeper that every request must pass through before it ever touches the database.

This is the architectural reason SOX/ITGC controls testing exists at all: if every rule and every access check is enforced by code running on the application server, then testing whether those controls work means testing the configuration and programs on that middle layer — not just “trusting” the database.

2.2 A Simple Analogy

Think of a bank branch. The client layer is the teller window and the paperwork you fill out — what you, the customer, actually see and touch. The application server is the bank's internal procedures and the staff who apply them: checking your ID, checking your balance, checking that the transaction is allowed, calculating interest, deciding whether a manager's approval is required. The database server is the vault and the ledger — the permanent, authoritative record of every account and every balance. You never walk into the vault yourself; you go through the teller, who follows procedure, who then instructs the vault.

2.3 One SAP System, Many Physical Computers

In a real SAP landscape, these three layers are rarely one computer each. A production system typically has:

- Many client devices — hundreds or thousands of laptops and phones running SAP GUI or a Fiori browser session.
- A cluster of application servers — several physical or virtual machines, each running many parallel “work processes” (explained in Chapter 4), so thousands of users can be served at once.
- One database server (or a small tightly-coupled cluster) — there is normally one logical database per SAP system, even though many application servers connect to it.

This matters for an auditor because it explains why access to the database layer directly (bypassing the application server's rule-checking) is treated as an extremely high-risk form of access — it is a way of getting into the vault without going through the teller and the bank's procedures.

Chapter 3 — The Client Layer

The client layer, also called the presentation layer, is anything a human being directly interacts with. In the SAP world, you will meet it in three common forms:

Form	What It Is
SAP GUI	The classic, dense, menu-and-transaction-code desktop application most experienced SAP users grew up on. Still widely used, especially for finance and back-office transactions.
SAP Fiori	SAP's modern, tile-based, browser-friendly interface, designed to look and behave more like a consumer app, and to work on phones and tablets as well as desktops.
Mobile / Browser Apps	Purpose-built apps (approvals, time entry, self-service) that talk to the same application server behind the scenes, just through a lighter front end.

3.1 What the Client Actually Does

The client's responsibilities are deliberately narrow:

- Render a screen — draw the fields, labels, buttons, and tables that the application server tells it to draw.
- Collect input — capture what the user types, clicks, or selects.
- Perform lightweight, local checks — for example, refusing to let you type letters into a field the server has flagged as “numbers only,” purely to save a wasted round trip.
- Package the input and send it — bundle up everything the user entered and transmit it to the application server for real processing.

3.2 What the Client Deliberately Does NOT Do

This is the point most newcomers get backwards, and it is important for controls thinking:

- The client does not decide whether you are authorized to do something — it may hide a button for convenience, but the real authorization check happens on the application server and is enforced there even if someone tried to bypass the client.
- The client does not store data permanently — close the window without saving, and — with rare local-draft exceptions — nothing you typed exists anywhere anymore.
- The client does not talk to the database directly — in a properly configured SAP landscape, the client has no direct path to the database server at all.

WHY AUDITORS CARE

Because the client cannot be trusted to enforce anything on its own, controls testing never accepts “the screen wouldn't let me” as evidence of a working control. A tester must confirm the restriction is

enforced on the application server (typically via an authorization object) — because a different client, a different tool, or a direct RFC/API call could otherwise skip the screen entirely.

Chapter 4 — The Application Server: SAP's Engine Room

This is the layer where SAP actually “thinks.” It is also the layer with the most new vocabulary, so we will build it up piece by piece.

4.1 The ABAP Stack

Most of what SOX/ITGC auditors deal with runs on SAP's application server built on a programming language and runtime environment called ABAP (Advanced Business Application Programming). Every custom program, every standard transaction, and every authorization check ultimately runs as ABAP code executing on the application server. You do not need to write ABAP — but you do need to know that “the program” behind any transaction is ABAP code sitting on this server, waiting to be triggered.

4.2 The Dispatcher — SAP's Traffic Cop

Every application server runs one dispatcher process. Its only job is to receive incoming requests from clients and hand each one to an available worker. Think of it as the host at a restaurant: it does not cook anything, it just decides which available table (worker) gets the next customer (request).

4.3 Work Processes — The Actual Workers

Behind the dispatcher sit a pool of work processes — the components that actually execute ABAP code. Each work process can only do one thing at a time, but an application server runs many of them in parallel so many users can be served “simultaneously” (really: very quickly taking turns). Work processes come in specialized types, and it is worth knowing the names because they appear constantly in system logs and in access-risk discussions:

Work Process Type	What It Does
Dialog (D)	Handles normal, interactive screen-by-screen work — what happens while you are actively using a transaction.
Update (V/U)	Handles the actual database “save” once a transaction is complete, often asynchronously, so the user is not stuck waiting on the database write.
Background (B)	Runs scheduled jobs with no user watching — batch jobs, nightly interfaces, mass reports.
Spool (S)	Manages print and output formatting.
Enqueue (E)	Manages locking — making sure two users cannot both change the same record at the same time and corrupt it.
Gateway / ICM	Handles web and RFC (remote function call) traffic — the entry point for Fiori, APIs, and system-to-system integration.

4.4 Where the Rules Live: Authorization Checks

This is the single most audit-relevant fact in this entire guide: every meaningful authorization check in SAP — “is this user allowed to do this, to this data, in this company code” — is code that runs on the application server, using an ABAP statement called `AUTHORITY-CHECK` against values stored in the user's security profile (roles, composite roles, and the authorization objects and field values inside them). This is why the SOX/ITGC course spends so much time on roles, authorization objects, and segregation of duties — that is where the actual gate lives.

4.5 Where the Rules Live: Business Logic

Beyond security, the application server is also where every business rule executes: that a journal entry must balance to zero, that a purchase order over a threshold needs a second approver, that inventory cannot go negative. In ABAP terms, this logic lives inside programs, function modules, BAPIs (Business APIs), and increasingly in newer development models — but architecturally, it is all “the middle box,” running before anything is allowed to reach the database.

Chapter 5 — The Database Server: Where the Record Lives

The database server layer is where SAP's data is permanently stored, organized into tables — grids of rows and columns, much like a well-structured spreadsheet, except formally governed by a database management system (DBMS) such as SAP HANA, Oracle, or SQL Server.

5.1 Tables, Rows, and Fields

A table has a fixed set of columns (fields) and an unlimited number of rows (records). For example, a simplified customer master table might have fields for Customer Number, Name, Country, and Credit Limit — and one row for every customer that has ever been created.

- Field — a single labeled piece of data, like “Credit Limit.”
- Record (row) — one complete set of field values — one customer, one vendor, one journal entry line.
- Table — the full collection of records of the same type.
- Primary key — the field (or combination of fields) that uniquely identifies each row, so the system never confuses two records.

5.2 SAP Has Thousands of Tables — and They Are Named, Not Random

A production SAP system has tens of thousands of tables. Many carry short technical names auditors encounter constantly — for example `USR02` (logon data), `AGR_USERS` (role assignments), `BKPF/BSEG` (accounting document header/line items), or `CDHDR/CDPOS` (change document header/detail, i.e. the system's own audit trail of who changed what, when). You will meet these properly inside the SOX/ITGC course; for now, the concept to hold onto is simpler: everything the system knows is a row in a table somewhere, and every table has a defined structure that never changes on the fly — which is precisely what makes reliable reporting and reliable auditing possible.

5.3 Relational Databases: Tables Talk to Each Other

SAP's database is relational, meaning tables are linked to one another through shared key fields rather than each table standing alone. A sales order table links to a customer table through the customer number; a journal entry table links to a chart-of-accounts table through the account number. This is what lets SAP answer questions like “show me every posting this vendor was involved in” by combining data from several tables at query time, instead of duplicating everything everywhere.

5.4 Why the Database Server Is Never Touched Directly (By Design)

In a properly governed SAP landscape, ordinary business users — and even most SAP support staff — have no direct access to the database. All changes are expected to flow through the application server, where business logic and authorization checks apply. Direct database access (sometimes via privileged DBA accounts, or emergency “firefighter” access) bypasses those checks entirely, which is exactly why it is treated as one of the highest-risk access types in an ITGC review, tightly restricted and heavily logged.

Chapter 6 — Program Components in a Client-Server Environment

Now that you know the three layers, it helps to see the actual building blocks SAP developers use to construct anything a user runs — a transaction, a report, an approval screen. You do not need to build these yourself; you need to recognize them by name.

Component	What It Is, in Plain English
Transaction (T-code)	A short code (like FB50 or ME21N) that acts as an address, pointing the system to a specific program and the correct starting screen for a business activity.
ABAP Program / Report	The actual compiled logic — the step-by-step instructions the application server executes.
Screen / Dynpro	“Dynamic Program” — SAP's term for a defined screen layout plus the logic that controls what happens when that screen is displayed and when it is submitted.
Module Pool	A structured collection of screens (dynpros) and their associated processing logic, chained together to form a multi-step transaction.
Function Module	A reusable, self-contained block of ABAP logic — a bit like a tool in a toolbox — that many different programs can call to perform a common task (e.g., “check credit limit”).
BAPI (Business API)	A special, standardized function module intended for external systems and integrations to call, following stricter rules for stability and compatibility.
Authorization Object	A named security “template” (e.g., F_BKPF_BUK for accounting documents by company code) that defines which fields are checked and what values are required for access to be granted.
Table / Structure	The data container itself (Chapter 5), plus in-memory “structures” that temporarily hold data while a program is running, before anything is saved.

A working SAP transaction is really an orchestration of several of these pieces at once: a T-code points to a module pool; the module pool displays a dynpro; the dynpro's logic calls function modules and BAPIs to validate and process data; authorization objects gate what the user is allowed to do at each step; and, when the transaction completes, the data is written to tables on the database server.

Chapter 7 — How SAP Really Works: The Journey From Keystroke to Database and Back

This chapter is the heart of this guide. We are going to trace, in order, exactly what happens between the moment a user presses Enter on their keyboard and the moment the confirmation message appears back on their screen. Read this section slowly — everything else in this guide is context for this one flow.

We will use a concrete, realistic example: a Finance user enters transaction FB50 to post a journal entry, fills in the accounting lines, and presses Save.

7.1 Before Any Business Data Is Even Typed: Logon

#	Where	What Happens
1	Logon request	The user opens SAP GUI (or a Fiori browser tab) and enters their user ID and password (or single sign-on credential). The client packages this and sends it to the application server they are configured to connect to.
2	Authentication	The application server checks the credential against the user's record in table USR02 (or an external identity provider if SSO is used). If it does not match, the session is rejected here — before anything else happens.
3	Session and buffer load	Once authenticated, the application server creates a user session and loads that user's authorization profile — every role, and every authorization object and value inside those roles — into memory, ready to be checked against every subsequent action for the life of that session.

7.2 Calling the Transaction

#	Where	What Happens
1	User types FB50 and presses Enter	The client captures the keystrokes as plain text input, does no interpretation of what “FB50” means, and sends the request to the application server.
2	Dispatcher hands off the request	The dispatcher on the application server receives the incoming request and assigns it to a free dialog work process.
3	T-code lookup	The work process looks up “FB50” in table TSTC (the transaction code table), which tells it exactly which program and initial screen (dynpro) is associated with that code.
4	First authorization check	Before displaying anything, the system runs an AUTHORITY-CHECK: does this user's loaded authorization profile include the right to even start transaction FB50 at all (authorization object S_TCODE)? If not, the user gets an authorization-failure message and the flow stops here.

7.3 Rendering the Screen (PBO — Process Before Output)

#	Where	What Happens
1	PBO logic runs	Every SAP screen (dynpro) has a defined “Process Before Output” routine — ABAP logic that runs on the application server just before the screen is sent to the user, to set default values, format fields, gray out fields the user should not touch, and so on.
2	Screen definition sent to client	The application server sends the fully prepared screen layout — fields, labels, current values, formatting — to the client. Note: it sends a description of the screen, not a live connection; the client then draws it.
3	Client renders	SAP GUI or the browser paints the actual pixels the user sees: input boxes, labels, buttons. The user is now looking at a screen, but nothing they do next has touched the application server or database yet.

7.4 The User Types Data

As the Finance user fills in the document date, company code, GL account, and amounts, the client is only capturing keystrokes into local screen fields and running lightweight local checks (for example, refusing non-numeric characters in an amount field). No business rule validation and no authorization check happens yet — the client is not equipped to do either. This is worth restating because it surprises people: you can type an amount that violates every business rule in the company, and the client will not stop you. Only the application server can.

7.5 The User Presses Enter or Save (PAI — Process After Input)

#	Where	What Happens
1	Data packaged and sent	The client bundles up everything the user entered on the screen and transmits it back to the application server as a structured request.
2	Dispatcher assigns a work process	As before, the dispatcher hands this request to an available dialog work process to execute the screen's “Process After Input” (PAI) logic.
3	Field-level and business validation	The application server runs validation logic — does the GL account exist? Is the company code valid? Do the debit and credit lines balance to zero? This is real ABAP code checking real rules, not cosmetic screen behavior.
4	Authorization checks for the actual action	Separately from the earlier “can you open this transaction at all” check, the system now runs further AUTHORITY-CHECK calls specific to the action being performed — for example, is this user authorized to post to this company code and this document type (authorization object F_BKPF_BUK)? If any required authorization is missing, the system blocks the posting and returns an authorization error, without writing anything to the database.
5	Business logic executes	Assuming validation and authorization pass, the underlying ABAP program (often calling shared function modules — the same posting logic used by many transactions) processes the entry: calculating totals, applying any workflow rules (e.g., does this need a second approver first?), and preparing the finished document.

7.6 Committing to the Database

#	Where	What Happens
1	Handoff to an update work process	Rather than making the user wait on the database itself, SAP typically hands the finished, validated transaction to a separate update work process, so the dialog work process is freed up immediately for the next user.
2	Database write (INSERT/UPDATE)	The update work process issues the actual database commands — for example, inserting new rows into BKPF (accounting document header) and BSEG (accounting document line items) — through the database's own SQL engine.
3	Locking, throughout	From the moment the document began processing, the enqueue work process held a lock on the relevant records so that no other user could simultaneously post a conflicting change to the same document or account; the lock is released once the database write completes.
4	Change logging	Where configured, the system also writes entries to change-document tables (CDHDR/CDPOS) or equivalent logs, recording who changed what, from what old value to what new value, and when — this becomes some of the most important raw evidence an ITGC auditor will later pull.
5	Database confirms the write	The database server confirms the commit succeeded (or reports an error, in which case the whole transaction is rolled back — nothing is left half-saved).

7.7 The Round Trip Back to the User

#	Where	What Happens
1	Application server prepares a response	A confirmation (“Document 1900000123 was posted in company code 1000”) or an error message is built by the application server.
2	Sent back through the dispatcher	The response is handed back through the same request/response channel to the client.
3	Client renders the result	SAP GUI or the browser displays the message to the user. Total elapsed time for this entire round trip — logon session already active — is typically well under a second.
THE ONE-SENTENCE VERSION		

#	Where	What Happens
		The client only ever displays and collects; the application server is where every rule, every calculation, and every authorization check actually happens, on every single action, every time — never just once at logon; and the database only ever stores what the application server has already validated and approved. Nothing reaches the database except through that gate.

Chapter 8 — What Is a T-Code, Really, and How Does It “Perform Its Magic”?

Once you have Chapter 7 in your head, a T-code stops being magic and becomes simple: it is a lookup key, nothing more. A transaction code is a short alphanumeric label (FB50, ME21N, SU01, PFCG) stored as a row in table TSTC, and that row points to the ABAP program and starting screen that should run when someone enters that code.

8.1 What a T-Code Is NOT

- It is not itself a program — it is a pointer to one.
- It does not itself carry any authorization — the authorization check (S_TCODE, plus whatever authorization objects the underlying program calls) happens separately, every time, based on the user's current role assignments.
- It does not guarantee a fixed set of screens — many transactions branch to different screens depending on data entered, configuration, and the user's authorizations (fields can be hidden, grayed out, or entire steps skipped based on what the user is allowed to see).

8.2 The Full Chain, Named

10. User enters a T-code → client sends it to the application server.
11. Application server looks up the T-code in table TSTC → finds the assigned program and screen.
12. System checks S_TCODE → is this user even allowed to start this transaction?
13. Program runs, dynpro is displayed (PBO) → client renders it.
14. User enters data → client collects it, does no validation of substance.
15. PAI logic runs on the application server → field checks, business rules, further authorization checks (specific authorization objects tied to that business process) all execute here.
16. If everything passes → update work process commits data to the relevant database tables, locks are released, and change logs are written.
17. Confirmation is sent back to the client and displayed to the user.

Every SAP transaction you will ever study in the SOX/ITGC course — posting a journal entry, creating a vendor, changing a user's role, releasing a purchase order — is this same chain, with different tables, different programs, and different authorization objects plugged in.

Chapter 9 — Why This All Matters: Connecting Architecture to Controls

Everything above was deliberately framed as “just how the system works,” with no auditing vocabulary forced in. This chapter makes the connections explicit, so the bridge into your SOX/ITGC coursework is obvious.

Architecture Fact	Why It Becomes a Control Topic
Client layer cannot be trusted to enforce anything	This is why testing a control never stops at “the screen prevented it” — testers must confirm enforcement exists at the application-server layer (an authorization object, a validation rule) that would hold even through a different client, an API call, or a batch upload.
Application server is where authorization checks live	This is why roles, authorization objects, and segregation-of-duties analysis are central topics — they are literally the configuration of the gate every transaction must pass through.
Update work processes and locking guarantee no half-saved data	This is part of why SAP's data can be relied on as a single source of truth for financial reporting — the architecture itself prevents partial, inconsistent saves.
Change-document tables (CDHDR/CDPOS) and similar logs	This is the system's own built-in audit trail, generated as a byproduct of the very save process you traced in Chapter 7 — it is primary evidence auditors pull to test whether changes were appropriate and properly authorized.
Direct database access bypasses the application server entirely	This is why DBA access, direct table maintenance (SM30/SE16 in change mode), and “firefighter” emergency access are treated as elevated-risk access requiring tight restriction, logging, and review — they skip the gate described in Chapters 4 and 7.
T-codes are just pointers, not security boundaries by themselves	This is why role design focuses on the authorization objects and field values behind a T-code, not merely on which T-codes appear in a menu — hiding a T-code from a menu does not remove the ability to run it.

In short: the SOX/ITGC course is going to ask you to evaluate whether the “gate” on the application server (Chapters 4 and 8) was correctly designed, correctly assigned to the right people, and correctly logged — for every meaningful transaction the company relies on. You cannot evaluate a gate you cannot picture. That is what this guide exists to give you.

Chapter 10 — Glossary of Terms

Term	Plain-English Meaning
ABAP	Advanced Business Application Programming — SAP's core programming language; the language most application-server logic is written in.
Application Server	The middle layer of the three-tier architecture; where business logic, calculations, and authorization checks actually execute.
AUTHORITY-CHECK	The ABAP command used inside programs to test whether the current user's authorizations satisfy the requirements to proceed.
Authorization Object	A named template of fields and values (e.g., F_BKPF_BUK) used to define and check a specific type of access right.
BAPI	Business API — a standardized function module designed for external systems to call into SAP.
Change Document (CDHDR/CDPOS)	SAP's built-in log of field-level changes: who changed what, from what value, to what value, and when.
Client (Presentation Layer)	The device/software the user directly interacts with — SAP GUI, Fiori, or a mobile app. Displays screens and collects input only.
Database Server	The layer where all data is permanently stored, organized into tables, managed by a database management system (e.g., SAP HANA).
Dialog Work Process	The type of work process that handles a user's interactive, screen-by-screen actions.
Dispatcher	The application-server component that receives incoming requests and assigns them to available work processes.
Dynpro	“Dynamic Program” — SAP's term for a defined screen plus its associated PBO/PAI processing logic.
Enqueue Work Process	The work process type responsible for locking records so simultaneous updates cannot corrupt data.
Function Module	A reusable block of ABAP logic that multiple programs can call to perform a common task.
Module Pool	A set of linked screens (dynpros) and logic that together form a complete transaction.
PAI (Process After Input)	The logic that runs on the application server after a user submits a screen — validation, business rules, authorization checks.
PBO (Process Before Output)	The logic that runs on the application server before a screen is displayed — setting defaults, formatting, field control.
Primary Key	The field (or set of fields) that uniquely identifies each row in a database table.
Relational Database	A database in which tables are linked to one another through shared key fields, rather than standing alone.

Term	Plain-English Meaning
Table	A structured collection of records (rows), each made up of a fixed set of fields (columns), stored on the database server.
T-Code (Transaction Code)	A short code (e.g., FB50) stored in table TSTC that points to the program and screen used to perform a specific business activity.
Three-Tier Architecture	The client / application server / database server model that structures how SAP (and most enterprise software) is built.
Update Work Process	The work process type that performs the actual database commit for a completed transaction, often asynchronously.
Work Process	A component on the application server that actually executes ABAP code; several specialized types exist (dialog, update, background, spool, enqueue).

Chapter 11 — Pre-Course Readiness Checklist

Before your first live SOX/ITGC Control Audit session, confirm you can do each of the following without looking back at this guide. If any item feels shaky, re-read the referenced chapter first — the live course moves quickly and assumes this foundation is already in place.

- I can draw and label the three-tier architecture (client / application server / database server) from memory. (Ch. 2)
- I can explain, in my own words, why the client cannot be trusted to enforce a business rule or a security restriction. (Ch. 3)
- I can name at least three types of application-server work processes and what each does. (Ch. 4)
- I can explain what an authorization object is and where the check for it actually happens. (Ch. 4, Ch. 8)
- I can explain what a table, a record, and a primary key are, in plain language. (Ch. 5)
- I can walk a colleague through the full journey of a transaction from keystroke to saved database record, step by step, without skipping the authorization checks. (Ch. 7)
- I can explain what a T-code actually is (a pointer, not a program, not a security boundary) and what the full chain behind it looks like. (Ch. 8)
- I can explain, for at least three architecture facts from Chapter 9, why that fact becomes something an ITGC auditor tests.
- I am comfortable with the glossary terms in Chapter 10 and can use them correctly in a sentence.

READY TO BEGIN

Once every box above is checked honestly, you have the working mental model this course track is built on. Proceed to the SAP SOX/ITGC Control Audit course series.